PROJECT SUBMISSION

Document Intelligence

A Retrieval-Augmented Generation platform for querying corporate policy documents with grounded answers and citations

Live Application	docintel-omega.vercel.app
GitHub Repository	github.com/bhavikachopra76/assignment_1
API Base URL	docintel-7vvc.onrender.com
Interactive API Docs	docintel-7vvc.onrender.com/docs

Bhavika Chopra · August 2026

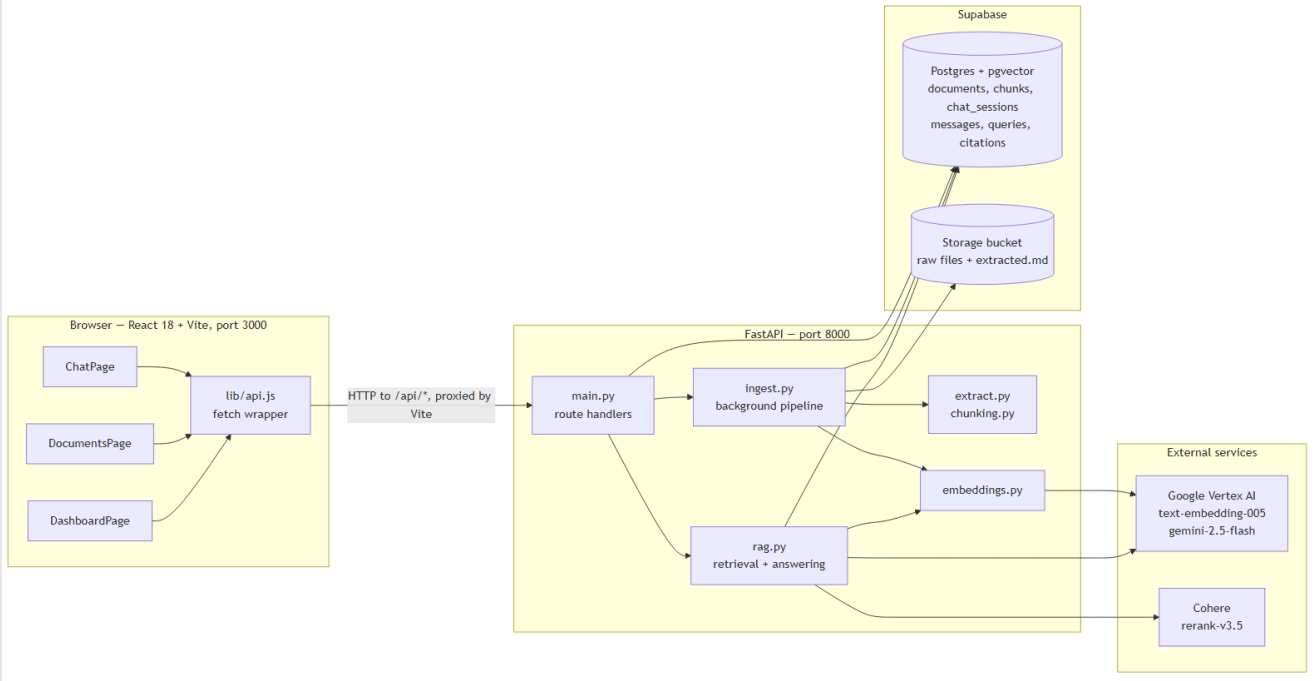
Technology Stack

Component	Technology
Frontend	React 18, Vite, Tailwind CSS — deployed on Vercel
Backend	FastAPI (Python) — deployed on Render
Document parsing	`PyMuPDF` / `pymupdf4llm`, `python-docx`
Chunking	Heading-aware Markdown splitting + `pysbd` sentence segmentation
Embeddings	Google Vertex AI `text-embedding-005` (768 dimensions)
Vector store	Supabase PostgreSQL with `pgvector`
Retrieval	Hybrid — dense vector similarity + sparse keyword full-text
Reranking	Cohere `rerank-v3.5`
Generation	Google Gemini `gemini-2.5-flash`

Part 1 — Architecture

NexaCore Document Intelligence is a Retrieval-Augmented Generation system for company policy documents. It has two pipelines that share the same storage layer: an **ingestion pipeline** that turns uploaded files into searchable vectors, and a **query pipeline** that answers questions using only those vectors.

1. System Overview

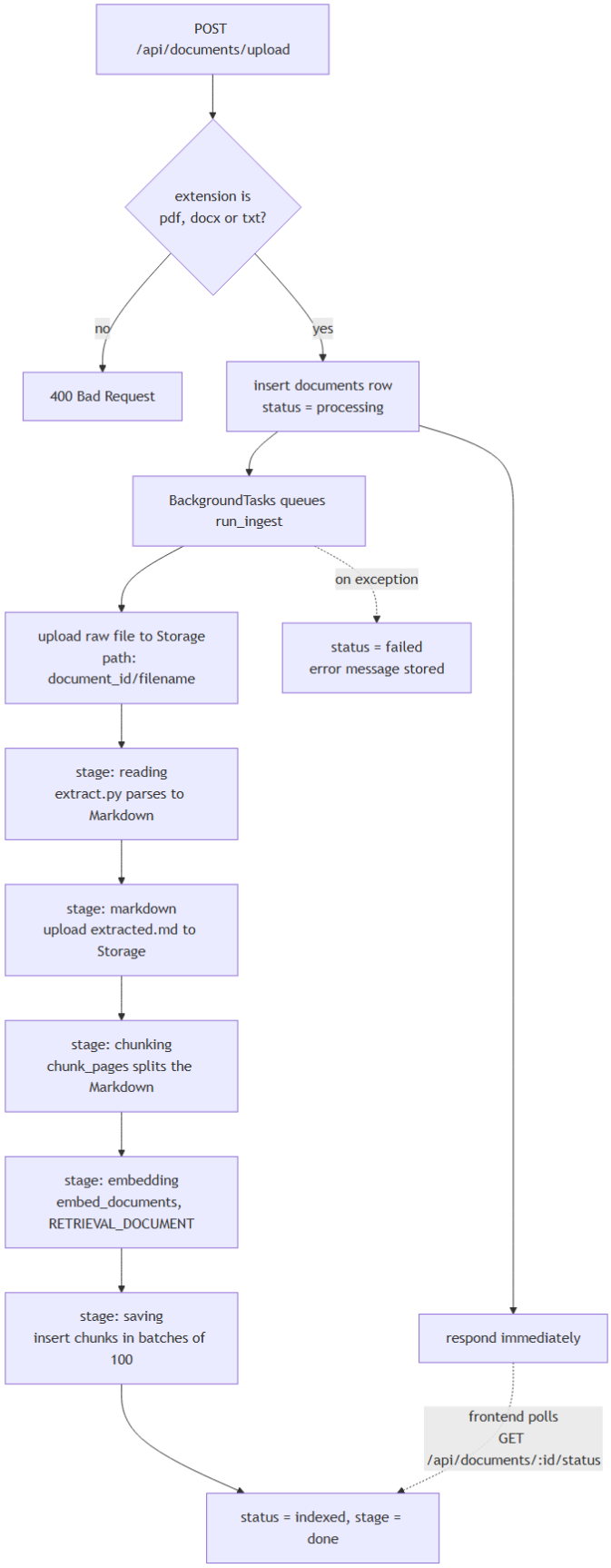


Frontend, API, database and model providers

The frontend never talks to Supabase or the model providers directly. Every call goes to `/api/*`, which the Vite dev server proxies to `http://localhost:8000` (`frontend/vite.config.js`). API keys stay on the backend.

2. Ingestion Pipeline

Uploading returns immediately. The actual work runs in a FastAPI `BackgroundTasks` job, and the frontend polls for progress.



Ingestion — upload through extraction, chunking, embedding and indexing

Extraction (`extract.py`)

Format	Library	What it produces
PDF	`pymupdf` + `pymupdf4llm`	Per-page Markdown, plus author and page count
DOCX	`python-docx`	Heading styles become `#` levels, tables become Markdown tables
TXT	standard library	`---` / `===` underlines are promoted to `##` headings

All three return the same shape — a list of `(page_number, text)` pairs — so everything downstream is format-agnostic. Only PDFs carry real page numbers; DOCX and TXT return `None`, which is why some citations show a page and some do not.

Chunking (`chunking.py`)

Splitting happens in two passes:

1. **Split on Markdown headings** so a chunk never spans two unrelated sections. The current heading is carried forward and stored with each chunk.
2. **Split on sentence boundaries** using `pysbd`, accumulating sentences until the chunk reaches roughly 600 tokens, then carrying the last 2 sentences into the next chunk as overlap.

A trailing chunk under 40 tokens is merged back into the previous one, so retrieval never returns a fragment too small to be useful.

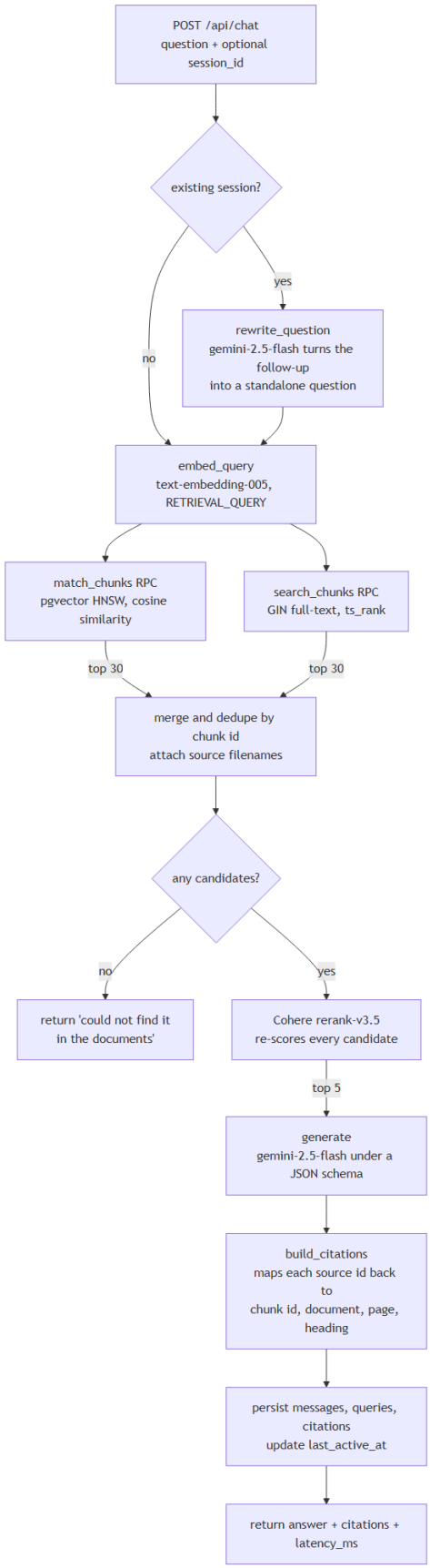
Indexing

Each chunk is stored with a 768-dimension embedding. The `fts` column is a **generated column** — Postgres computes the `tsvector` on insert, so keyword search needs no extra write from the application:

```
fts tsvector generated always as (to_tsvector('english', content)) stored
```

Two indexes back the two retrieval channels: an HNSW index with cosine distance for vector search, and a GIN index for full-text search.

3. Query Pipeline



Query — question through hybrid retrieval, reranking and grounded answering

Why hybrid retrieval

Dense vector search matches meaning but misses exact tokens — policy numbers, allowance amounts, specific job titles. Keyword search catches those but misses paraphrases. Running both and merging gives up to 60 candidates with the recall of both methods.

The two searches are issued sequentially, and the sparse leg is skipped entirely when the question contains no term longer than two characters. Results are merged into a dict keyed by chunk id, so a chunk found by both channels appears once.

Why rerank

60 candidates is far more context than the model needs, and precision matters more than recall once the answer is being written. Cohere `rerank-v3.5` scores every candidate against the question directly — a cross-encoder, unlike the bi-encoder embeddings used for retrieval — and only the top 5 reach the prompt.

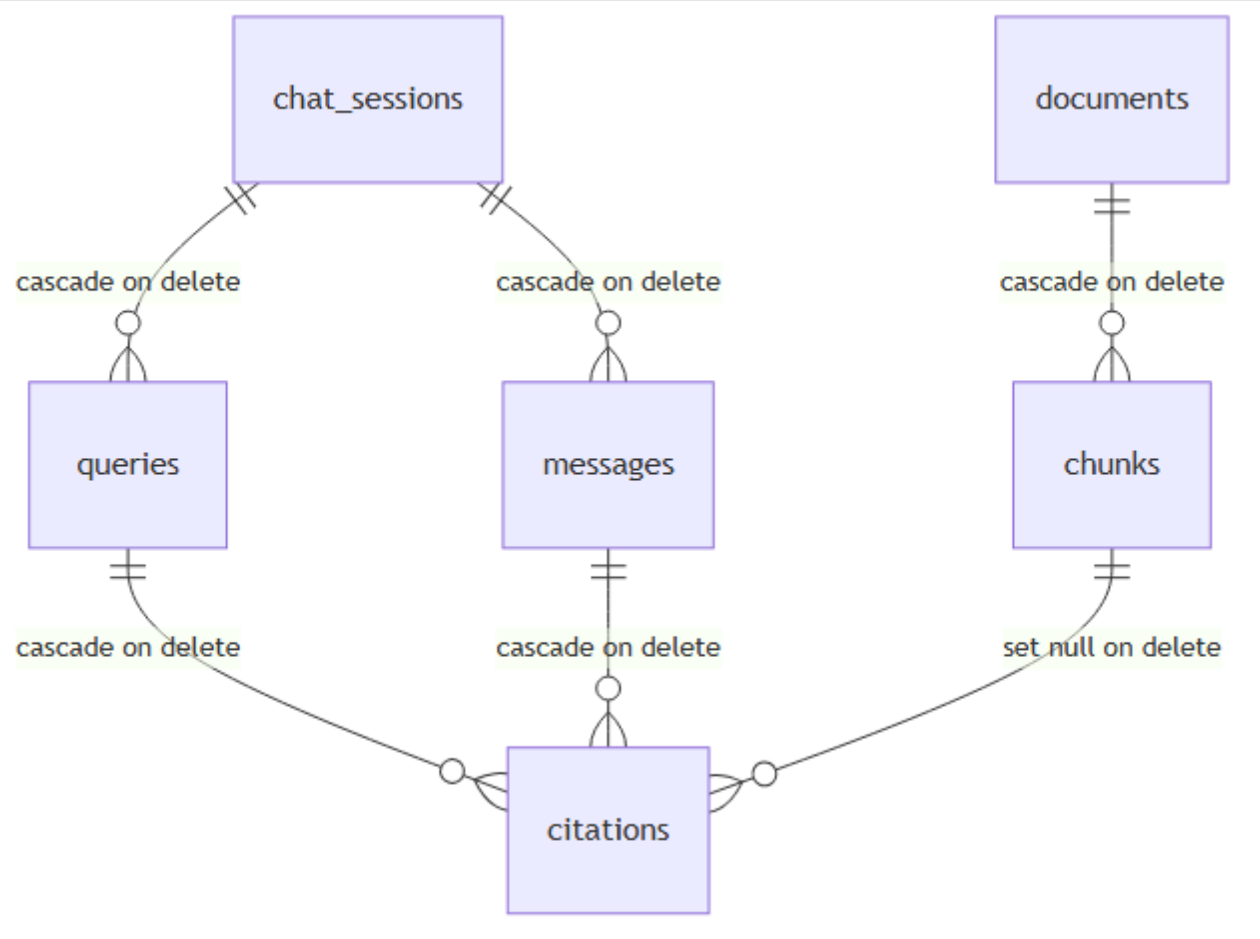
Grounded answering

The prompt numbers each source and requires the model to reply in a fixed JSON shape:

```
{"answer": "...", "citations": [{"id": 0, "quote": "copied word for word"}]}
```

The schema is enforced by Vertex AI's `response_schema`, not just requested in the prompt. `build_citations` then validates each returned id against the actual candidate list and discards anything out of range, so a hallucinated source number cannot produce a citation. The model is instructed to answer only from the sources and to say it could not find the answer otherwise.

4. Data Model



Tables and their cascade-delete relationships

Table	Holds
`documents`	File metadata, ingestion `status` and `stage`, page and chunk counts
`chunks`	Chunk text, heading, page number, 768-d `embedding`, generated `fts`
`chat_sessions`	One row per conversation, ordered by `last_active_at`
`messages`	User and assistant turns, used to rebuild a conversation
`queries`	One row per question with `latency_ms`, powering the dashboard
`citations`	Verbatim quote plus a link back to the chunk it came from

Deleting a document cascades to its chunks. Citations keep their stored quote and document name even after the chunk is gone, because `chunk_id` is set to null rather than deleted — so old answers stay readable in the dashboard.

5. Request Lifecycle Summary

Endpoint	Path through the system
`POST /api/documents/upload`	Validate → insert row → queue background job → respond
`GET /api/documents/:id/status`	Read `documents` row, used for progress polling
`DELETE /api/documents/:id`	Remove Storage files, delete row, chunks cascade
`POST /api/chat`	History → rewrite → embed → hybrid search → rerank → generate → persist
`GET /api/chat/sessions`	20 most recent sessions by `last_active_at`
`GET /api/dashboard/stats`	Aggregate counts and average latency over all queries
`GET /api/dashboard/responses`	Recent queries joined to their first citation

6. Failure Handling

Rate limits are the main failure mode, since both embedding and generation call quota-limited APIs.

- `embed_batch` retries up to 5 times on a 429, sleeping 20 seconds between attempts.
- `call_gemini` retries up to 3 times on a 429 with a backoff of 10, then 20 seconds.
- If generation still fails, `/api/chat` returns **503** with a readable message rather than a stack trace.
- Any exception during ingestion is caught by `run_ingest`, which marks the document `failed` and stores the error, so a bad file never leaves a row stuck on `processing`.

Embedding batches are capped at 100 texts or roughly 18,000 tokens, whichever comes first, to stay inside the Vertex AI request limit.

Part 2 — API Documentation

REST API for the NexaCore Document Intelligence platform.

Base URL	`https://docintel-7vvc.onrender.com`
Interactive docs	/docs (Swagger UI) · /redoc
Machine-readable spec	/openapi.json
Local	`http://localhost:8000`

The Swagger page is executable — you can upload a document and ask a question from the browser without writing any client code.

> The API is hosted on Render's free tier and spins down after 15 minutes of inactivity. The first request after an idle period can take up to two minutes while the server wakes, so give it a generous timeout. Every request after that is normal speed.

Conventions

- All responses are JSON. Request bodies are JSON except the upload endpoint, which is `multipart/form-data`.
- Response fields are **camelCase**, while the underlying database columns are snake_case. `schemas.py` does that translation, so `file_type` becomes `type` and `uploaded_at` becomes `uploadDate`.
- Timestamps are ISO 8601 UTC. `uploadDate` is date-only (`YYYY-MM-DD`).
- Sizes, latencies and relative times are **pre-formatted strings** built for display (`"41 KB"` , `"1.8s"` , `"5 min ago"`), not raw numbers.
- **There is no authentication.** Every endpoint is public. This is a known limitation — the service is scoped as a single-tenant demo, and an API key layer or Supabase Auth would be the first addition before any real deployment.

Endpoints

Method	Path	Purpose
`GET`	`/`	Health check
`GET`	`/api/documents`	List all documents
`POST`	`/api/documents/upload`	Upload one or more documents
`GET`	`/api/documents/{document_id}/status`	Check ingestion progress
`DELETE`	`/api/documents/{document_id}`	Delete a document
`POST`	`/api/chat`	Ask a question
`GET`	`/api/chat/sessions`	List recent chat sessions
`GET`	`/api/chat/sessions/{session_id}/messages`	Get messages in a session
`DELETE`	`/api/chat/sessions/{session_id}`	Delete a chat session
`GET`	`/api/dashboard/stats`	Overall usage statistics
`GET`	`/api/dashboard/queries`	Recent questions
`GET`	`/api/dashboard/responses`	Recent answers with sources

Health

`GET /`

```
{ "status": "ok", "docs": "/docs" }
```

Documents

`GET /api/documents`

All uploaded documents, newest first.

```
[
  {
    "id": "5d36f10e-80e3-49fb-8e54-6225fbabae64",
    "filename": "NexaCore_Travel_Policy.docx",
    "type": "DOCX",
    "size": "41 KB",
    "pages": null,
    "chunks": 76,
    "uploadDate": "2026-08-04",
    "status": "Indexed",
    "stage": "done",
    "author": "NexaCore HR",
    "error": null
  }
]
```


`pages` is `null` for DOCX and TXT — only PDFs carry page numbers, which is also why some citations show a page and others don't.

`POST /api/documents/upload`

Uploads one or more files. Content type `multipart/form-data`, field name `files`, repeated once per file.

Accepted extensions: **pdf**, **docx**, **txt**.

```
curl -X POST https://docintel-7vvc.onrender.com/api/documents/upload \
  -F "files=@NexaCore_Travel_Policy.docx" \
  -F "files=@NexaCore_Employee_Handbook.pdf"
```

200 — returns immediately, before indexing has run:


```
[
  {
    "id": "8c1f2b40-1d3e-4a77-9f21-6b0c5d2e7a13",
    "filename": "NexaCore_Travel_Policy.docx",
    "type": "DOCX",
    "size": "41 KB",
    "pages": null,
    "chunks": 0,
    "uploadDate": "2026-08-05",
    "status": "Processing",
    "stage": null,
    "author": null,
    "error": null
  }
]
```


The response arrives before the document is searchable. Parsing, chunking and embedding run in the background — poll the status endpoint below.

400 — if *any* file has an unsupported extension, the whole request is rejected and nothing is uploaded:

```
{ "detail": "notes.pptx must be a PDF, DOCX or TXT file" }
```

`GET /api/documents/{document_id}/status`

Same shape as a single document from the list endpoint. Poll this while `status` is `Processing`.

Ingestion lifecycle:

status	stage	Meaning
`Processing`	`reading`	Extracting text from the file
`Processing`	`markdown`	Saving the extracted Markdown
`Processing`	`chunking`	Splitting into passages
`Processing`	`embedding`	Generating vectors
`Processing`	`saving`	Writing chunks to the database
`Indexed`	`done`	Ready to query
`Failed`	*(last stage reached)*	`error` holds the reason

The document only becomes searchable at `Indexed`.

404 — `{ "detail": "Document not found" }`

`DELETE /api/documents/{document_id}`

Removes the stored files and the database row. Chunks are removed by cascade.

```
{ "deleted": "5d36f10e-80e3-49fb-8e54-6225fbabae64" }
```

Citations that referenced this document keep their stored quote and document name, so previously given answers stay readable in the dashboard.

404 — `{ "detail": "Document not found" }`

Chat

`POST /api/chat`

Answers a question using only the uploaded documents.

Request:

```
{
  "question": "What is the travel expense reimbursement limit?",
  "session_id": null
}
```

Field	Type	Required	Notes
`question`	string	yes	Cannot be empty or whitespace
`session_id`	string | null	no	Omit to start a new conversation

Pass `session_id` back on later questions to keep context. When present, the last 6 messages are used to rewrite an ambiguous follow-up ("what about international?") into a standalone question before searching.

200:

```
{
  "sessionId": "9f3c1a02-77b5-4c8e-a1d6-3e0b9c4f2a8d",
  "message": {
    "id": "b21e7d54-9a08-4f13-8c2b-5d7e1f60a934",
    "sender": "assistant",
    "text": "Employees may claim up to $75 per day for meals while travelling domestically.",
    "timestamp": "2026-08-05T11:24:03.512Z",
    "citations": [
      {
        "id": "c47a9e13-2f60-4b85-91d3-8a2c6e0f5b71",
        "documentName": "NexaCore_Travel_Policy.docx",
        "pageNumber": null,
        "section": "Meal Allowances",
        "originalParagraph": "Employees travelling domestically may claim up to $75 per day for
meals."
      }
    ]
  }
}
```

`originalParagraph` is copied verbatim from the source passage, so a citation can be checked against the original document.

If nothing relevant is found, the answer says so and `citations` is empty — the model is instructed never to answer from outside knowledge.

400 — `{ "detail": "Question cannot be empty" }`

404 — `{ "detail": "Chat not found" }` if `session_id` is given but no longer exists.

503 — upstream AI rate limit, after internal retries:

```
{ "detail": "Too many requests to the AI service right now. Try again in a minute." }
```

> Typical response time is 3–8 seconds: question rewriting, embedding, two searches, reranking and generation happen in sequence. On the free tier, expect longer.

`GET /api/chat/sessions`

The 20 most recently active conversations.

```
[
  {
    "id": "9f3c1a02-77b5-4c8e-a1d6-3e0b9c4f2a8d",
    "title": "What is the travel expense reimbursement limit?",
    "lastActive": "5 min ago"
  }
]
```

`title` is the first question, truncated to 60 characters.

`GET /api/chat/sessions/{session_id}/messages`

Full history for one conversation, oldest first. Same message shape as `POST /api/chat`, with `sender` being `user` or `assistant`. User messages always have empty `citations`.

Returns `[]` for an unknown session.

`DELETE /api/chat/sessions/{session_id}`

```
{ "deleted": "9f3c1a02-77b5-4c8e-a1d6-3e0b9c4f2a8d" }
```

Messages, queries and citations for the session are removed by cascade, so its questions also disappear from dashboard analytics.

404 — `{ "detail": "Chat not found" }`

Dashboard

`GET /api/dashboard/stats`

```
{
  "totalDocuments": 3,
  "totalChunks": 326,
  "totalQuestions": 7,
  "avgResponseTime": "24.2s",
  "documentGrowth": "+3 this week",
  "chunkGrowth": "+326 chunks",
  "questionGrowth": "+2 today",
  "timeTrend": "over 7 queries"
}
```

The four `*Growth` / `timeTrend` fields are display strings for the stat cards, not values to compute with.

`GET /api/dashboard/queries`

The last 50 questions.

```
[
  {
    "id": "e83b6c19-4d2a-4f70-b5e1-9c7a3f08d264",
    "question": "What is the travel expense reimbursement limit?",
    "time": "5 min ago",
    "status": "Completed",
    "latency": "1.8s"
  }
]
```

`GET /api/dashboard/responses`

The last 50 answers, each labelled with the first document it cited.

```
[
  {
    "id": "e83b6c19-4d2a-4f70-b5e1-9c7a3f08d264",
    "question": "What is the travel expense reimbursement limit?",
    "answer": "Employees may claim up to $75 per day for meals while travelling domestically.",
    "source": "NexaCore_Travel_Policy.docx",
    "timestamp": "2026-08-05T11:24:03.512Z"
  }
]
```

`source` includes a page when one exists (`"Handbook.pdf (Page 12)"`) and is `"-"` when the answer had no citations.

Errors

Every error uses FastAPI's standard shape:

```
{ "detail": "Document not found" }
```

Code	When
`400`	Empty question, or an unsupported file extension
`404`	Document or chat session does not exist
`422`	Request body failed validation (missing or wrong-typed field)
`503`	Upstream AI rate limit, after internal retries

An id that is not a valid UUID is treated as not found rather than as a server error, so a malformed `document_id` or `session_id` returns the same **404** shape as one that simply does not exist.

Rate limits are retried before surfacing: embedding retries 5 times with 20-second waits, generation retries 3 times backing off 10 then 20 seconds. A `503` means those were exhausted.

Failures during ingestion never surface as HTTP errors, because upload returns before that work runs. They appear as `status: "Failed"` with a reason in `error` on the document.

External Services

The API calls three third-party services. Their own documentation is authoritative; this table records what each is used for.

Service	Used for	Model / feature	Docs
Google Vertex AI	Embedding chunks and queries	`text-embedding-005`, 768 dimensions	docs
Google Vertex AI	Question rewriting and grounded answering	`gemini-2.5-flash` with an enforced JSON response schema	docs
Cohere	Reranking retrieved candidates	`rerank-v3.5`, 60 candidates down to top 5	docs
Supabase	Postgres with pgvector, plus file storage	HNSW cosine index, GIN full-text index	docs

Chunks are embedded with task type `RETRIEVAL_DOCUMENT` and questions with `RETRIEVAL_QUERY` — the same model, but asymmetric task types produce better matching between short questions and longer passages.